Assignment 2: Write-Up

Parva Shaileshkumar Patel (psp8502)

Collaboration and AI use statement

I did not work with other students on the solution but I discussed the assignment's general expectations with classmates, to confirm that my approach was in line with them. I used Claude Code in VS Code throughout. It explained the design document, the code and the hardware concepts, walking me through each part with questions I answered before moving on. For each of the four modules and the pricing change, it explained the specification and the unit tests, I predicted the logic and it proposed code in the chat. I wrote it into the files myself, reviewed line by line and tested (unit tests, `make verify`, `make timing`). It analysed the timing reports and critical-path logs, proposed pipelining pricing and the second cut point. It also suggested the seed runs that measured placement noise. It also wrote study notes and a reference version of these answers, from which I wrote this write-up and it reviewed my draft and suggested corrections.

Explain the change you made and how it allows us to increase the clock frequency.

The design's clock is limited by its slowest register-to-register path because every register shares one clock. make timing showed that pricing was the only module under 30 MHz. It was 18.88 MHz on its own and 18.84 MHz for the whole design. Every other module ran at 42.76 MHz or more. nextpnr's critical-path report showed that pricing does its whole calculation in one cycle, one step after another. It computes the spread, the spread in ticks S, $S\times$ bid size, fifteen comparisons to find fair value's tick, then the tick prices and the credit. In total that's 53 ns in a single cycle and the two steps that compare and keep the last comparison that holds take about 33 ns. I split pricing into two pipeline stages, with a register after the multiply, the point closest to halfway. Each half now gets its own clock cycle, so the clock period only has to cover the longer half, about 30 ns, instead of the whole 53 ns chain. My first cut reached only 26.56 MHz. The fifteen multiples k $\times$ total size had been computed in parallel, off the critical path and the cut moved them in front of the comparisons. Moving them into stage 1 and storing all fifteen in the new register, made stage 2 start directly at the comparisons. The whole design now runs at 33.46 MHz and between 31.6 and 34.6 MHz across ten placement seeds. Therefore it's above 30 MHz for all of them. I checked that make verify still passes for all ten scenarios as the math is unchanged and also pricing keeps no state between items. The cost here is one more cycle of latency. We now have 10 clock cycles instead of 9 and about 670 extra flip-flops. But each cycle is now much shorter. So an order leaves after about 299 ns instead of 478 ns. Throughput is unchanged because items still arrive at most every two cycles. I chose pipelining over the alternatives. A divider for the microprice would be larger and slower still. A third stage would add another cycle without being needed. A multicycle-path constraint isn't part of this flow and would depend on the merge never sending items faster. Pricing's second stage is still the slowest path. Its fifteen comparison results are always ones followed by zeros, so the boundary could be found in one step. This would be as the book does, instead of a chain of fifteen. After pricing, the next limit is about 42 MHz which are the quote manager and the book.

Explain how your implementation of the order book differs from the C++ implementation of the order book. How does the implementation of each order book play to the strengths of the respective platform?

The C++ book keeps every price. A ladder with one slot per tick with 200,000 per side. It holds each level's size and a 16 MB array indexed directly by order reference holds each order. Here finding a level or an order is a single memory access. the best bid & ask are cached, so reading them is free; only when the best level empties does the book scans the ladder for the next non empty tick. This is where the CPU's strengths lies as memory is plentiful and cheap. also, caches make nearby accesses fast & a loop of varying length is cheap when it's usually short. **The FPGA book** keeps only the best 16 levels per side which is sorted and stored in flip-flops. here, the per order state lives separately in a 4,096 entry block RAM. Every update is compared with all 16 levels at once. Each level then takes its new value from itself, a neighbour or the new level. Therefore, a size change or an insert or a delete all finish in one cycle. Reading the top of book costs nothing as it's slot 0 and wired to the output. This is the strength as it provides parallel logic and fixed timing. Moreover, it sidesteps the FPGA's weaknesses. Storage on the chip is scarce which is about 470 KB of block RAM in total. So keeping every price level is not feasible. a block RAM serves only one access per port per cycle and a search of varying length would create a long & slow path. Sixteen levels held in flip-flops avoid both problems. But that choice has two costs. The first is capacity which have issues such as having a seventeenth level or an order whose table slot is taken by a newer one is lost; both lead to a sticky error and the quote manager stops quoting. Whereas the C++ book, with room for a million orders and every tick up to $2,000, carries on. The second is area: the 16 level book takes 12,330 LUTs, 64% of the design's logic, to make every update finish in a single cycle.